

Lab 5 Exam

准备工作：创建并切换到 lab5-exam 分支

请基于已完成的 lab5 提交自动初始化 lab5-exam 分支，然后在开发机依次执行：

```
$ cd ~/学号
$ git fetch
$ git checkout lab5-exam
```

初始化的 lab5-exam 分支基于课下完成的 lab5 分支，并且在 tests 目录下添加了 lab5-ls 样例测试目录。

题目背景 & 题目描述

在 Linux 中，ls（单词 list 的简写）是一条常用命令，可以显示指定工作目录下的内容（列出目前工作目录所含的文件及子目录）。附带参数的 ls -a 可以显示出包含隐藏文件在内的所有内容。

在本次测试中，我们希望实现一个简易的目录列出服务函数 `int file_list(const char *path, u_int all, struct Ls_res *res)`。该函数支持列出无通配符绝对路径 path 对应的目录下的所有文件的名字（当参数 all 非 0 时额外列出以 . 开头的隐藏文件，不用考虑也不用输出 ./ 和 ../，无需递归列出子目录中的文件），并记录文件数量（当 all 非 0 时计入隐藏文件数量），将结果存入 res 指向的预定义结构体。额外地，你需要区分非目录文件和目录文件：**对于目录文件，你需要在目录文件原有名字的后面增加一个 / 字符来区分。**

预定义的存放结果的结构体如下（后面“实现思路”部分还会再次给出）：

```
struct Ls_res {
    char file_name[MAXLSNUM][MAXLSENTRY];
    int count;
};
```

为了确保你的理解正确，这里给出一个例子。对于以下的目录结构，

```
/
├─ .hidden/
│   └─ hidden_file
├─ dir1/
│   ├── dir1_file1
│   └─ dir1_file2
├─ dir2/
│   └─ dir2_file
├─ file1
└─ .hidden_file
```

使用 `file_list(path="/", all=0, res)` 后，res 中的内容如下：

```
// file_name 中的顺序不影响评测
// 这里的“=”表示“是”而不是“赋值”
res = {
    file_name[0] = "dir1/"
    file_name[1] = "dir2/"
    file_name[2] = "file1"
    count = 3
}
```

而使用 `file_list(path="/", all=1, res)` 后, `res` 中的内容如下:

```
// file_name 中的顺序不影响评测
// 这里的“=”表示“是”而不是“赋值”
res = {
    file_name[0] = ".hidden/"
    file_name[1] = "dir1/"
    file_name[2] = "dir2/"
    file_name[3] = "file1"
    file_name[4] = ".hidden_file"
    count = 5
}
```

更为具体的行为规则与实现提示如下。

1. 关于 `all` 参数:

- 当 `all != 0` 时, 列出所有非空目录项 (包括以 `.` 开头的隐藏文件);
- 当 `all == 0` 时, 不列出以 `.` 开头的文件名。

2. 关于普通文件与目录文件的区分:

- 如果为目录文件, 在 `res` 记录该文件名字时末尾添加 `/` 以示区分 (例如 `dir/`);
- 如果为普通文件, 只记录名字 (例如 `file`)。

3. 在识别文件名时, 你需要**跳过空名文件** (文件名以 `\0` 开头的文件, 这表示它们是磁盘块中还未被使用的文件控制块或被删除了的文件对应的文件控制块) 以保证计数正确。

4. 测试点保证你无需考虑找到的指定目录下文件名过长的问题。

5. `Ls_res` 结构体成员 `file_name` 应当从 `file_name[0]` 开始填充 (无需考虑目录下的文件名顺序问题), 成员 `count` 用于记录本次请求得到的文件数量, **否则不保证评测正确**。

6. 函数返回值 (错误码与课程约定保持一致):

- 路径不存在或找不到: `-E_NOT_FOUND`
- 路径非法 (空或过长): `-E_BAD_PATH`
- 正常执行: 返回 `0`

7. 测试点保证: 测试点保证当 `path` 能找到时一定是目录, 且 `path` 以 `/` 结尾; `path` 以无通配符的绝对路径给出。

实现思路

1. 在 `user/include/lib.h` 的 `#define pages ((const volatile struct Page *)UPAGES)` 定义后添加查询结果结构体:

```
#define MAXLSENTRY (MAXNAMELEN + 2)
#define MAXLSNUM ((PAGE_SIZE - sizeof(int)) / MAXLSENTRY)
struct Ls_res {
    char file_name[MAXLSNUM][MAXLSENTRY];
    int count;
};
```

2. 在 `user/include/lib.h` 的 `// file.c` 部分添加声明:

```
int file_list(const char *path, u_int all, struct Ls_res *res);
```

3. 将 `file_list` 函数的实现添加到 `user/lib/file.c`:

```
int file_list(const char *path, u_int all, struct Ls_res *res) {
    return fsipc_ls(path, all, res);
}
```

4. 在 `user/include/fsreq.h` 中的枚举类型中增加一个对于文件系统的请求类型 `FSREQ_LS`, 请注意要把它放在 `MAX_FSREQNO` 前;

5. 在 `user/include/fsreq.h` 的 `#endif` 之前添加请求结构体:

```
struct Fsreq_ls {
    char req_path[MAXPATHLEN];
    u_int req_all;
};
```

6. 在 `user/include/lib.h` 的 `// fsipc.c` 部分新增声明:

```
int fsipc_ls(const char *path, u_int all, struct Ls_res *res);
```

7. 在 `user/lib/fsipc.c` 中添加 `fsipc_ls` 函数的实现:

```
int fsipc_ls(const char *path, u_int all, struct Ls_res *res) {
    if (path[0] == '\0' || strlen(path) >= MAXPATHLEN) {
        return -E_BAD_PATH;
    }

    struct Fsreq_ls *req = (struct Fsreq_ls *)fsipcbuf;
    strcpy((char *)req->req_path, path);
    req->req_all = all;

    return fsipc(FSREQ_LS, req, res, 0);
}
```

8. 在 `fs/serv.c` 的服务函数分发表 `serve_table` 最后新增一项:

```
[FSREQ_LS] = serve_ls
```

9. 在 `fs/serv.c` 中添加 `serve_ls` 函数的实现:

```
void serve_ls(u_int envid, struct Fsreq_ls *rq) {
    struct Ls_res res __attribute__((aligned(PAGE_SIZE))) = {};
    int r = list_files(rq->req_path, rq->req_all, &res);

    if (r) {
        ipc_send(envid, r, 0, 0);
    } else {
        ipc_send(envid, 0, (const void *)&res, PTE_D | PTE_LIBRARY);
    }
}
```

10. 在 `fs/serv.h` 中添加声明：

```
int list_files(const char *path, u_int all, struct Ls_res *res);
```

11. 在 `fs/fs.c` 中完成 `list_files` 函数，实现本服务的核心功能。

实现提示

可以在 `fs/fs.c` 中构建一个辅助函数 `list_dir_entries` 用于实现核心功能（遍历目录下的所有文件，将信息填入 `res`）：

```
int list_dir_entries(struct File *ls_dir, u_int all, struct Ls_res *res) {
    u_int nblock;
    nblock = ls_dir->f_size / BLOCK_SIZE;

    for (int i = 0; i < nblock; i++) {
        void *blk;
        try(file_get_block(ls_dir, i, &blk));
        struct File *files = (struct File *)blk;

        for (struct File *f = files; f < files + FILE2BLK; ++f) {
            // 按照“行为规则”表述，对当前磁盘块中的每一个文件控制块 f，
            // 完成函数的核心功能。具体功能请参考题面要求。
            // lab5-exam: Your code here. (1/3)

        }
    }
    return 0;
}
```

然后调用辅助函数实现目标函数功能：

```

int list_files(const char *path, u_int all, struct Ls_res *res) {
    struct File *ls_dir;
    char path1[MAXPATHLEN];
    strcpy(path1, path); // 将 const char* 转为 char*

    // 找到 path 对应的目录。提示：可能会用到 walk_path。
    // lab5-exam: Your code here. (2/3)

    // 调用辅助函数，完成核心功能。
    // lab5-exam: Your code here. (3/3)

}

```

样例输出 & 本地测试

对于测试样例文件

```

#include <lib.h>
struct Ls_res res __attribute__((aligned(PAGE_SIZE)));
struct Ls_res ans __attribute__((aligned(PAGE_SIZE))) = {
    .file_name = {
        "file_1",
        "file_2"
    },
    .count = 2
};
int check_id __attribute__((aligned(PAGE_SIZE)));

void sort_res(struct Ls_res *res) {
    int cnt = res->count;
    char tmp[MAXLSENTRY];
    for (int i = 0; i < cnt - 1; i++) {
        for (int j = 0; j < cnt - i - 1; j++) {
            if (strcmp(res->file_name[j], res->file_name[j + 1]) > 0) {
                strcpy(tmp, res->file_name[j]);
                strcpy(res->file_name[j], res->file_name[j + 1]);
                strcpy(res->file_name[j + 1], tmp);
            }
        }
    }
}

void check_res(struct Ls_res *res, struct Ls_res *ans) {
    check_id++;
    sort_res(res);
    sort_res(ans);

    // check: count
    if (res->count == ans->count) {
        debugf("check_%d: \'count\' correct!\n", check_id);
    } else {
        debugf("check_%d: \'count\' wrong!!!\n", check_id);
        return;
    }
}

```

```

// check: file_name
for (int i = 0; i < ans->count; i++) {
    if (strcmp(res->file_name[i], "") == 0) {
        debugf("check_%d: one \'file_name\' is empty!\n", check_id);
        return;
    }
}
for (int i = 0; i < ans->count; i++) {
    if (strcmp(res->file_name[i], ans->file_name[i]) != 0) {
        debugf("check_%d: one \'file_name\' is wrong!\n", check_id);
        return;
    }
}
debugf("check_%d: \'file_name\' correct!\n", check_id);
}

int main() {
    file_list("/bin/", 0, &res);
    check_res(&res, &ans);
    return 0;
}

```

和测试样例文件结构

```

/
└─ bin/
   └─ file1
   └─ file2

```

应当看到以下输出：

```

check_1: 'count' correct!
check_1: 'file_name' correct!

```

你可以使用 `make test lab=5_ls && make run` 在本地测试上述样例。

如果需要自行设置样例或更改样例评测输出，可以修改 `tests/lab5_ls/` 中 `rootfs/` 中的文件结构、`ls_check.c` 中的评测标准与评测输出。

提交评测 & 评测标准

请在开发机中执行下列命令后，在课程网站上提交评测：

```

$ cd ~/学号/
$ git add -A
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push

```

测试点说明及分数分布如下：

测试点序号	评测说明	分值
1	样例	5
2	仅包含非隐藏目录	15
3	仅包含非隐藏目录和非隐藏文件	15
4	仅包含非隐藏文件和隐藏文件	15
5	综合测试	50

测试点依赖关系如下（箭头方向表示依赖方向）：

